



Master 1 SD, SIME

Master 2 SIME

Apprentissage profond



Nom: Coelho Fernandes

Prénom: Rodrigo César

Séance de TP 4
Explicabilité – Segmentation Multi-classes avec UNet

QUESTIONS THÉORIQUES

1- Définir le principe de la segmentation.

Semantic segmentation aims to create a mask where each pixel of an image is individually classified into a specific class, allowing the identification of elements present in the original image and the spaces they occupy.

2- Expliquer le rôle de l'encoder et du décodeur de l'architecture Unet.

The encoder transforms the image into a feature vector by extracting characteristics from the original image. It uses a series of convolutional and pooling layers that progressively reduce the spatial dimensions of the image while increasing the number of feature channels.

The decoder transforms this vector into a mask where each pixel of the image is classified. It reconstructs the vector to match the original image's dimensions. It implements an upsampling function (Transpose Convolution) and residual connections to preserve high-resolution details.

3- Quels sont les facteurs qui peuvent rendre difficile la segmentation multi-classe ? (3 facteurs minimum)

Firstly, the presence of complex boundaries between objects. When different classes share similar or overlapping edges, the model may struggle to accurately distinguish between them, leading to misclassification.

Another important factor is image quality. Low resolution, noise, or blurriness can significantly impact the model's ability to correctly detect and segment objects. Poor-quality images may lack the necessary details, making it difficult for the model to learn and differentiate features effectively.

Also, excessive or insufficient lighting can distort the visual information in an image. Overexposure can wash out important details, while underexposure can make objects difficult to detect, ultimately affecting the accuracy of the segmentation process.

QUESTIONS SUR LE CODE

- 4- Lors du resize des images et des masques quelles sont les techniques d'interpolations utilisées et pourquoi différent-elles ?

Image.BILINEAR interpolation is applied to the images, because it provides smoother resizing by performing a weighted average of the surrounding pixels. This helps maintain smooth transitions and visual details without introducing unwanted artifacts.

On the other hand, for the masks a Image.NEAREST interpolation is applied, that is, a nearest neighbor interpolation. If a method like bilinear interpolation were used for masks, it would introduce intermediate values that could distort the annotation by creating non-existent classes. Thus, nearest-neighbor interpolation ensures that each pixel in the mask retains its original class after resizing.

- 5- Dans UNET.py, que signifie ***nn.BatchNorm2d(output_channel)*** ?

This function is used within the Convblock class to apply batch normalization to the output of a convolutional layer. It is a technique used to normalize the output of a previous activation layer by subtracting the batch mean and dividing by the batch standard deviation, which helps in reducing internal covariate shift. Essentially, it ensures that the network doesn't have to relearn the correct data distributions during training.

- 6- Expliquer la ligne de code : **`scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3, verbose=True)`**

ReduceLROnPlateau is PyTorch's learning rate scheduler. This automatically reduces the learning rate of the optimizer when the model's performance (on a given metric) plateaus during training. If the metric continues to plateau, the learning rate will be further reduced. This helps improve training by dynamically adjusting the learning rate to avoid getting stuck in local minimal.

Parameters of the function:

optimizer: This is the optimizer to which the scheduler will be applied (e.g., torch.optim.SGD or torch.optim.Adam).

mode='min': means that the training is monitoring a metric (like loss) that should decrease. If the loss stops decreasing and remains constant for a certain number of epochs (determined by patience), the learning rate will be reduced.

factor=0.5: This is the factor by which the learning rate will be reduced.

patience=3: This parameter specifies the number of epochs to wait for the monitored metric to improve before reducing the learning rate.

verbose=True: True, PyTorch will print messages every time the learning rate is reduced.

QUESTIONS PRATIQUES

7- Pourquoi le premier masque généré est uniforme ?

At the beginning of the training, an attempt is made to load the parameters of a saved model. However, since there is no saved model during the first execution, the result appears uniform, and the training starts “from scratch.”

Suivre les 3 expériences :

Expérience 1: NEW_SIZE = (80,80), USE_SCHEDULER = False

Expérience 2: NEW_SIZE = (80,80), USE_SCHEDULER = True

Expérience 3: NEW_SIZE = (256,256), USE_SCHEDULER = True

8- Pour chaque expérience, commenter l'évolution qualitative des masques générés toutes les 10 époques. (capture d'écran + commentaires)

From the training timeline shown below, we can see that the first two training sessions reduce the original image to 80x80 pixels. This means that the decoder is unable to fully restore the details of the original image, making it difficult to accurately represent the different classes. Both experiments end with an image that represents the major elements of the original scene as well as possible, such as the street, sidewalk, and buildings in the background. Experiment 2 performs better since it does not make the mistake of introducing a blue class in the upper center of the image, staying more faithful to the expected result.

As the epochs progress, the results evolve from an image with generalized classifications to one with more details and refined shapes, signaling the learning process.

Only Experiment 3 successfully identifies small details in the original image, such as pedestrians, thanks to its higher final resolution. This inclusion of details translates into a lower loss: at the end of the training, the validation loss is 0.0222 for the first two experiments and 0.0148 for the last one. Similarly, the learning process starts with a highly generalized representation that gradually gains detail and adjusts the distribution of elements as training progresses.

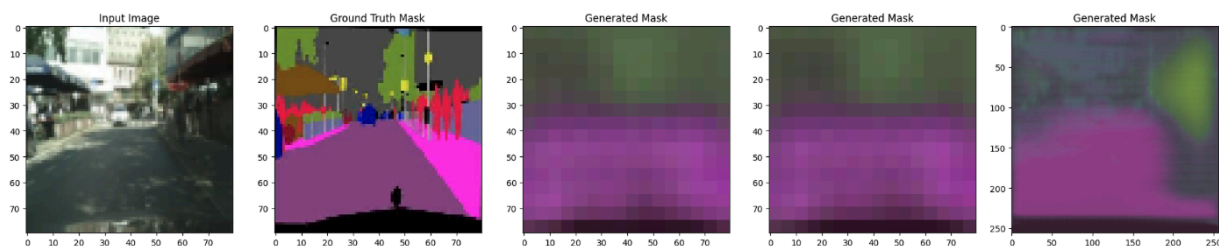
époque

Expérience 1
NEW_SIZE = (80,80),
USE_SCHEDULER = False

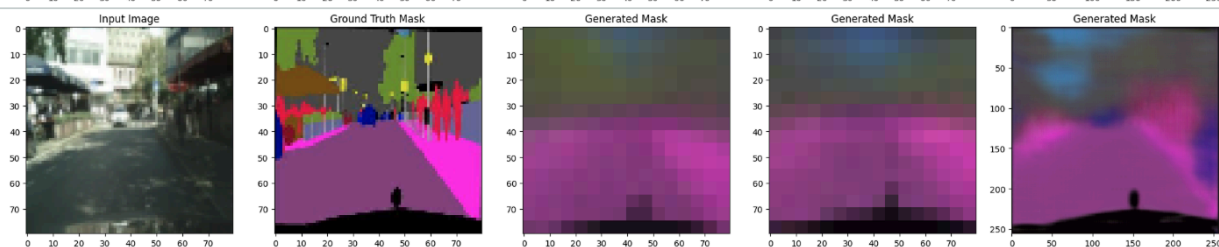
Expérience 2
NEW_SIZE = (80,80),
USE_SCHEDULER = True

Expérience 3
NEW_SIZE = (256,256),
USE_SCHEDULER = True

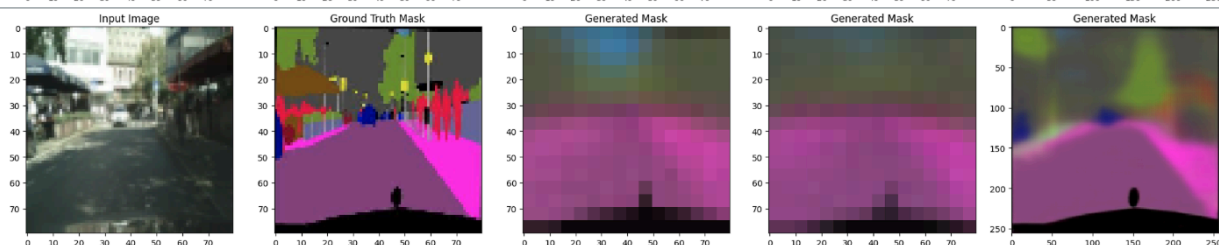
0



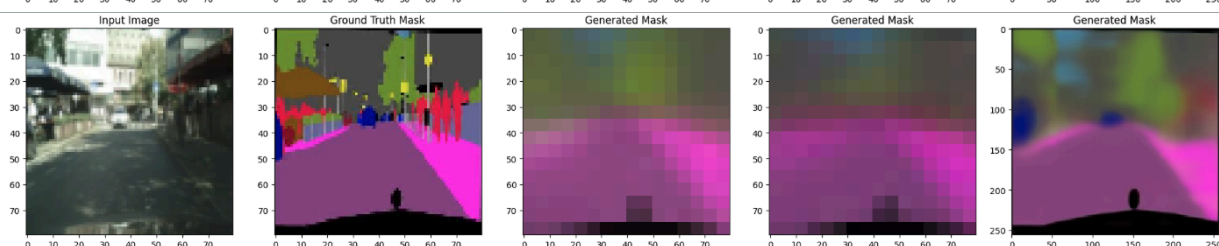
9



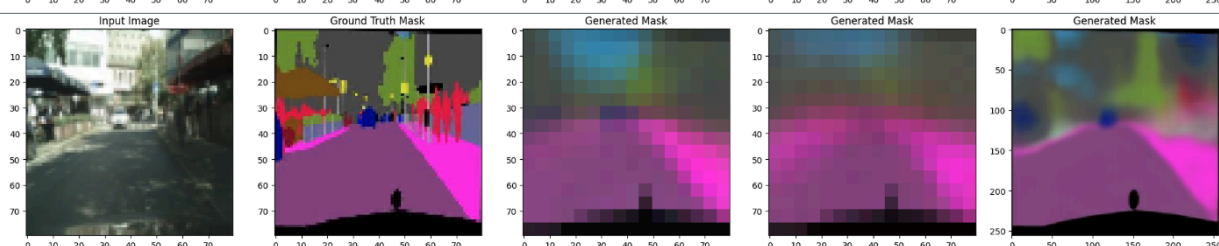
18



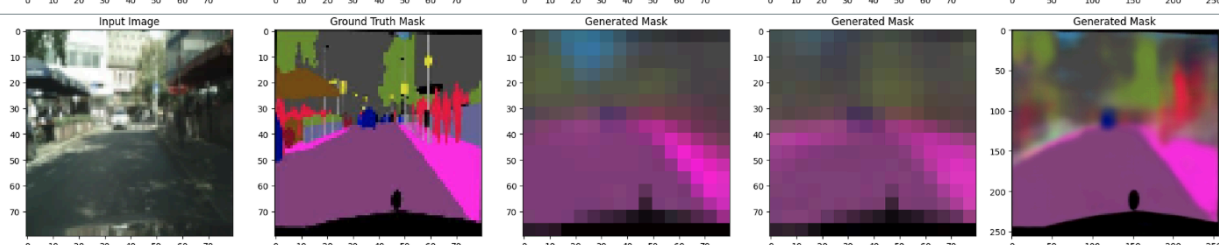
21



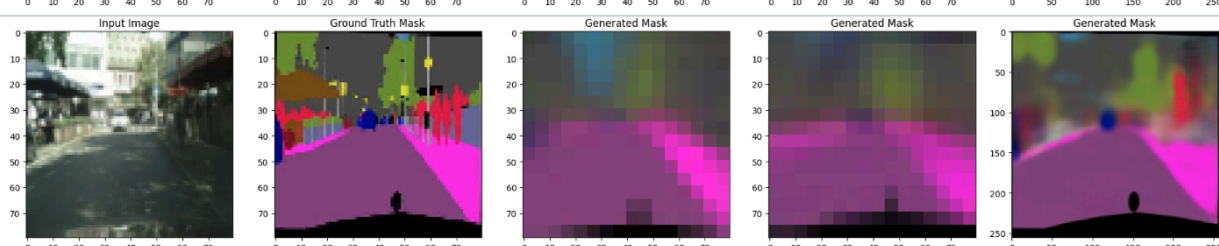
30



39

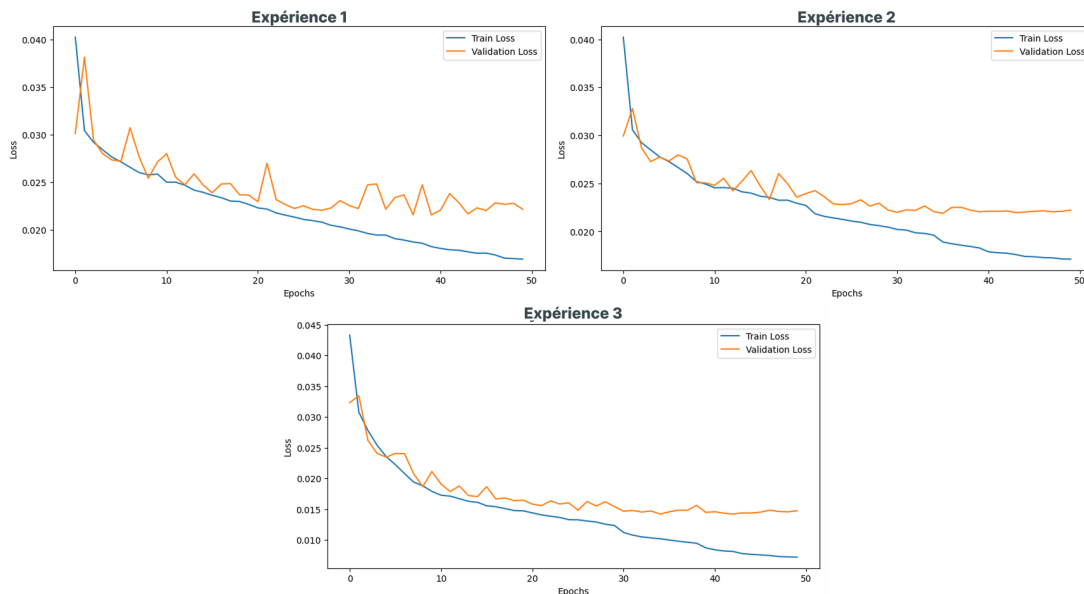


48



9- Quelle est l'impact de l'ajout du scheduler sur l'apprentissage ?

We can observe the effect of the scheduler through the loss graph as a function of epochs. When the scheduler is inactive, as in Experiment 1, the validation loss remains unstable, indicating that the learning rate is consistently high. In the other two cases, we observe two moments where the validation loss stabilizes, followed by sharp drops, signaling changes in the learning rate.



However, it is important to mention that if the scheduler is not properly configured, it can negatively impact training. If the learning rate is reduced too much, the model may make little progress after its activation.

10-Quelles techniques pourriez-vous mettre en place pour améliorer l'apprentissage du modèle ? (3 techniques minimum)

To improve the model's learning, an essential approach is to test different values for hyperparameters such as `BATCH_SIZE`, `LEARNING_RATE`, and `WEIGHT_DECAY` in order to find the best combination that maximizes the network's performance. Additionally, it is crucial to properly set the Scheduler's factor to avoid any negative effects.

Another strategy to enhance the results would be to increase the resolution of the masks used in training to ensure more accurate predictions. However, this adjustment also significantly raises the computational cost, requiring a balance between quality and training efficiency. To mitigate this effect, applying Early Stopping would be a good idea.

Furthermore, data augmentation techniques, such as mirroring and adjusting brightness or contrast, could be employed to make the model more robust.